

March 11, 2026

1 SARSA en Taxi-v3

Estudio del algoritmo de Diferencia Temporal (TD) On-Policy

Este notebook implementa y analiza el algoritmo **SARSA** (State-Action-Reward-State-Action) sobre el entorno **Taxi-v3** de Gymnasium. A diferencia de los métodos de Monte Carlo, SARSA utiliza **bootstrapping**, permitiendo al agente aprender de sus estimaciones actuales sin esperar al final del episodio.

1.0.1 SARSA

- **Aprendizaje On-Policy:** El agente aprende sobre la política que está ejecutando actualmente (incluyendo la exploración ϵ -greedy).
- **Diferencia Temporal (TD):** Actualiza los valores Q basándose en la recompensa inmediata y la estimación del siguiente estado-acción:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- **Eficiencia en Taxi-v3:** Con 500 estados y penalizaciones severas (-10 por acciones ilegales), el aprendizaje paso a paso de SARSA permite evitar errores críticos mucho antes que los métodos de episodio completo.

1.1 1. Preparación del Entorno

La preparación consta de las siguientes partes: - **Instalación de dependencias:** librería gymnasium. - **Importación de librerías:** numpy, matplotlib, tqdm. - **Creación del entorno Taxi-v3:** espacio de observación Discrete(500), espacio de acciones Discrete(6), render_mode='ansi' para visualización en texto. - **Importación de clases del repositorio:** SARSAAgent, funciones de visualización.

1.2 ##### Código de la Instalación e Importación

```
[3]: %%capture
#@title Instalamos gymnasium
!pip install gymnasium
```

```
[4]: #@title Importamos librerías
import sys
sys.path.insert(0, 'src')
```

```

import os
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
import gymnasium as gym

from agents import SARSAAgent
from plotting import plot, plot_lengths, show_greedy_episode, print_q_summary,
    plot_policy_taxi, plot_comparison, plot_lengths_comparison

```

```

[5]: # @title Semilla para reproducibilidad (sección 5.4 del PDF)
# Patrón recomendado: fijar la misma semilla en NumPy, Python y Gymnasium.
# Sin PyTorch porque este notebook es tabular (no usa redes neuronales).
SEED = 2024

# Fijar la semilla en NumPy
np.random.seed(SEED)
np.random.default_rng(SEED)

# Fijar la semilla en Python (evita variabilidad en hashing)
os.environ['PYTHONHASHSEED'] = str(SEED)

print(f"Semilla fija: SEED = {SEED}")

```

Semilla fija: SEED = 2024

```

[6]: #@title Importamos el entorno Taxi-v3
env = gym.make('Taxi-v3', render_mode='ansi')
env.reset(seed=SEED) # Fija la semilla del entorno (patrón sección 5.4 del
    PDF)

nS = env.observation_space.n # 500
nA = env.action_space.n # 6
print(f"Estados: {nS}, Acciones: {nA}")
print(f"Recompensas: -1 por paso | +20 entrega exitosa | -10 acción ilegal")
print()
# Estado inicial de ejemplo
obs, info = env.reset(seed=SEED)
print(f"Estado inicial (seed={SEED}): {obs}")
taxi_row, taxi_col, pass_loc, dest_idx = env.unwrapped.decode(obs)
locs = ['R', 'G', 'Y', 'B', 'en taxi']
print(f" Taxi: ({taxi_row},{taxi_col}), Pasajero: {locs[pass_loc]}, Destino:
    {locs[dest_idx]}")
print(env.render())

```

Estados: 500, Acciones: 6

Recompensas: -1 por paso | +20 entrega exitosa | -10 acción ilegal

Estado inicial (seed=2024): 333
 Taxi: (3,1), Pasajero: B, Destino: G

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+
```

1.3 2. Diseño del Agente

El diseño del agente sigue el esquema de Gymnasium (sección 5.2): `__init__`, `get_action`, `update`, `stats`.

1.3.1 Política Epsilon-Greedy

- El agente utiliza una política ϵ -**soft** para garantizar la exploración continua del espacio de estados.
- Con probabilidad $1 - \epsilon$, selecciona la acción con el mayor valor Q (explotación).
- Con probabilidad ϵ , selecciona una acción al azar (exploración).

1.3.2 Algoritmo SARSA

- Implementado en la clase `SARSAAgent`, este algoritmo actualiza la tabla Q en cada paso del entorno.
- A diferencia de Q-Learning, SARSA es más conservador ya que considera la acción real que el agente tomará en el siguiente paso (la cual podría ser una acción exploratoria arriesgada).

```
[7]: # @title Constantes del entorno Taxi-v3
# Acciones del entorno Taxi-v3
SOUTH, NORTH, EAST, WEST, PICKUP, DROPOFF = 0, 1, 2, 3, 4, 5
ACTION_NAMES = {0: 'S↓', 1: 'N↑', 2: 'E→', 3: 'W←', 4: 'PU', 5: 'DO'}
# La política greedy se obtiene con agent.pi_star(seed=SEED,
↪action_names=ACTION_NAMES)
```

1.4 3. Experimentación

1.4.1 3.1 Representaciones Gráficas

Gráfica 1 — Recompensa media acumulada: $f(t) = \frac{\sum_{i=1}^t R_i}{t}$. En Taxi-v3, buscamos que esta métrica supere los valores negativos iniciales y se estabilice en valores positivos (indicativo de entregas exitosas consistentes).

Gráfica 2 — Longitud de los episodios: Mide cuántos pasos tarda el taxi en completar la tarea. 1. **Inicio:** Los episodios suelen truncarse al límite de pasos (200) debido a la exploración

ciega. 2. **Aprendizaje:** La longitud cae drásticamente a medida que el agente aprende a recoger al pasajero y evitar muros. 3. **Convergencia:** Se estabiliza cerca del óptimo teórico (aprox. 12-15 pasos).

Función `show_greedy_episode`: Permite observar el comportamiento final del taxista sin exploración, verificando si ha aprendido la secuencia lógica: Ir al pasajero $\rightarrow$ Pickup $\rightarrow$ Ir al destino $\rightarrow$ Dropoff.

Función `plot_policy_taxi`: Visualiza la política óptima aprendida, mostrando las acciones preferidas en cada estado. En Taxi-v3, esto revela las rutas óptimas hacia el pasajero y el destino, así como las acciones de pickup y dropoff.

1.4.2 3.2 SARSA en Taxi-v3

Se entrenan **100 000 episodios** con ε con decaimiento: $\varepsilon = \min(1.0, 1000/(t + 1))$ igual que en MC para poder comparar después resultados.

El decaimiento es necesario porque: - Al inicio, ε alto favorece exploración amplia (el taxi aprende a evitar penalizaciones). - Al final, $\varepsilon \rightarrow 0$ hace que la política de comportamiento se acerque a la greedy y los episodios sean más cortos y eficientes.

La semilla numpy fija (`np.random.seed(42)`) garantiza reproducibilidad.

```
[8]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=0.2, alpha=0.1, discount_factor=1.0,
    ↪decay=True)
agent_S.train(num_episodes=100000)
list_stats_S, list_lengths_S = agent_S.stats()
Q_S = agent_S.q_values
```

```
11%|          | 10571/100000 [00:09<00:32, 2779.11it/s]
success: -112.7662, epsilon: 0.1000

21%|          | 20573/100000 [00:12<00:24, 3204.17it/s]
success: -54.2963, epsilon: 0.0500

30%|          | 30362/100000 [00:15<00:21, 3315.13it/s]
success: -34.2654, epsilon: 0.0333

40%|          | 40428/100000 [00:18<00:17, 3340.20it/s]
success: -24.0718, epsilon: 0.0250

51%|          | 50625/100000 [00:21<00:14, 3429.85it/s]
success: -17.8989, epsilon: 0.0200

60%|          | 60479/100000 [00:24<00:11, 3454.89it/s]
success: -13.7467, epsilon: 0.0167

71%|          | 70672/100000 [00:27<00:08, 3440.72it/s]
success: -10.7666, epsilon: 0.0143
```

```

81%|      | 80624/100000 [00:30<00:05, 3387.12it/s]
success: -8.5152, epsilon: 0.0125

91%|      | 90523/100000 [00:33<00:02, 3301.58it/s]
success: -6.7591, epsilon: 0.0111

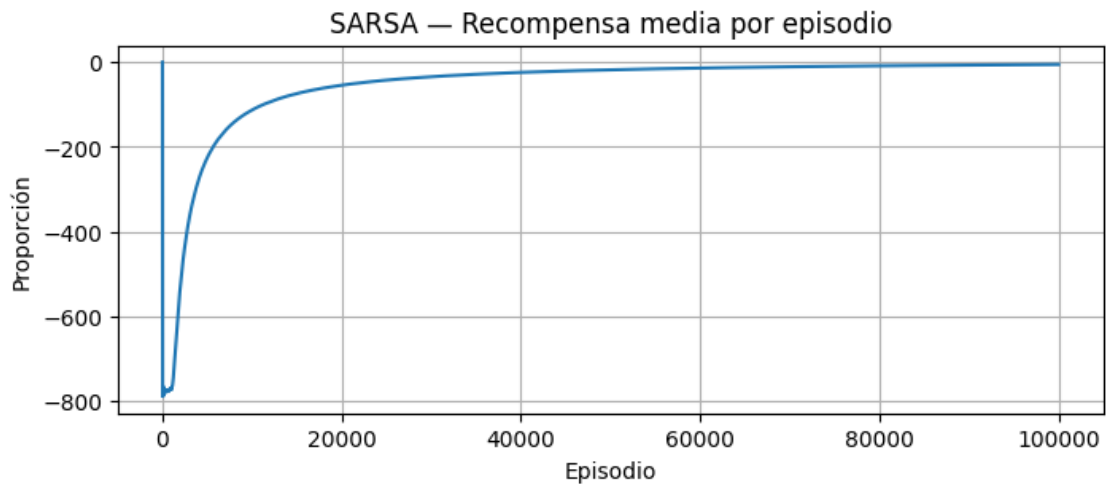
100%|     | 100000/100000 [00:36<00:00, 2736.74it/s]
success: -5.3416, epsilon: 0.0100

```

```

[9]: #@title Recompensa media por episodio (on-policy)
plot(list_stats_S, title='SARSA - Recompensa media por episodio')
print(f'Recompensa media final: {list_stats_S[-1]:.2f}')

```

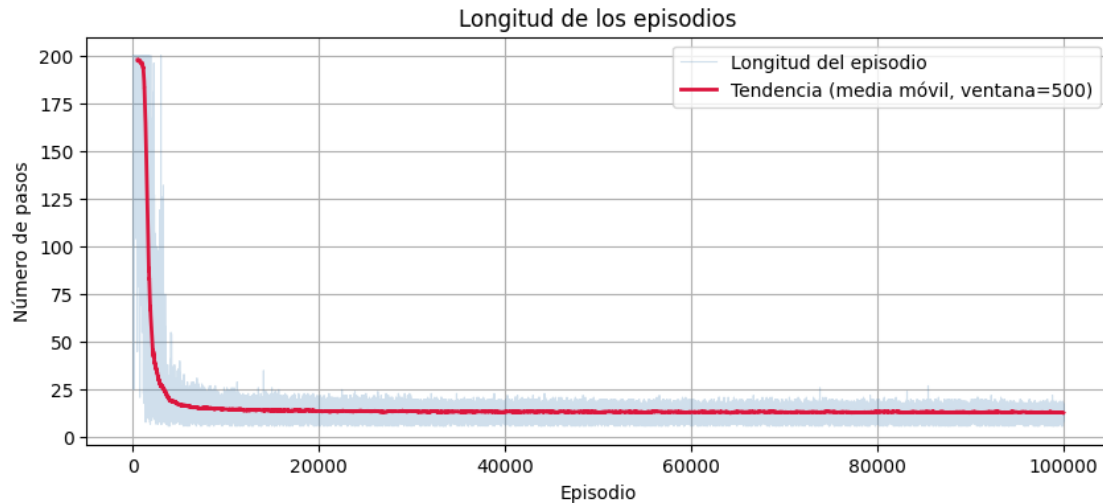


Recompensa media final: -5.34

```

[10]: #@title Longitud de episodios (on-policy)
plot_lengths(list_lengths_S)
print(f'Longitud media final (últimos 1000 episodios): {np.
↪mean(list_lengths_S[-1000:]):.2f} pasos')

```



Longitud media final (últimos 1000 episodios): 13.04 pasos

Resumen estadístico de la tabla Q aprendida. Con 500 estados y 6 acciones la tabla tiene 3000 entradas. Se muestran estadísticas globales y algunos estados decodificados para verificar que el agente ha aprendido acciones coherentes.

```
[11]: # @title Resumen tabla Q - on-policy
print_q_summary(env, Q_S, title="On-Policy - Tabla Q")

--- On-Policy - Tabla Q ---
Entradas no nulas : 2400/3000 (80.0%)
Max Q              : 20.000
Min Q (no nulo)   : -66.155
Q media (no nulo)  : -19.264

Muestra de estados:
[ 0] taxi=(0,0) pas=R      dst=R  -> mejor=S Q=[0. 0. 0. 0. 0. 0.]
[124] taxi=(1,1) pas=G     dst=R  -> mejor=S Q=[ 3.66 -41.49 -48.88 -37.28
-51.09 -45.92]
[249] taxi=(2,2) pas=Y     dst=G  -> mejor=W Q=[ -7.93 -18.77 -2.31  6.
-7.83 -14.03]
[374] taxi=(3,3) pas=B     dst=Y  -> mejor=S Q=[10.8  4.08  5.37  6.58
-1.04 -1.2 ]
[499] taxi=(4,4) pas=en taxi dst=B -> mejor=W Q=[-2.21 -2.36  3.85 18.99
-9.73 -6.71]
```

Se muestra la política óptima greedy obtenida. `agent.pi_star(seed, action_names)` sigue $\arg\max Q[s]$ desde el estado inicial fijo (semilla SEED) y devuelve la secuencia de acciones en formato legible ($S \downarrow N \uparrow E \rightarrow W \leftarrow PU DO$). También se ejecuta `show_greedy_episode` para verificar visualmente el resultado en formato ANSI.

```
[12]: # @title Política final (on-policy)
pi, actions = agent_S.pi_star(seed=SEED, action_names=ACTION_NAMES)
print("Política óptima obtenida (on-policy)")
print(f"Acciones: {actions}")
print()
show_greedy_episode(env, Q_S, seed=SEED, title="SARSA - Episodio greedy")
```

Política óptima obtenida (on-policy)

Acciones: N↑ → E→ → E→ → S↓ → S↓ → PU → E→ → N↑ → N↑ → N↑ → N↑ → DO

SARSA - Episodio greedy | Recompensa: 9 | Pasos: 12 | Éxito

Estado inicial:

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+
```

Estado final:

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+
```

(Dropoff)

```
[13]: state, info = env.reset(seed=SEED)
# Extraer: taxi_row, taxi_col, pass_loc, dest_idx
_, _, pass_loc, dest_idx = env.unwrapped.decode(state)

plot_policy_taxi(env, agent_S.q_values, passenger_loc=pass_loc,
    ↪ destination_idx=dest_idx)
```

Estado Inicial del Entorno (Pasajero: 3, Destino: 1):

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
```

+-----+

Mapa de Política (Mejor acción por celda):

```
-----  
| ↓ | ↓ | → | ↓ | ← |  
| → | ↓ | ↓ | ↓ | ↓ |  
| → | → | → | ↓ | ↓ |  
| ↑ | ↑ | ↑ | ↓ | ↓ |  
| ↑ | → | ↑ | P | ← |  
-----
```

1.4.3 3.3 Modificación de parámetros de SARSA

Los hiperparámetros en el algoritmo SARSA son fundamentales para equilibrar la explotación versus la exploración y cómo se gestionan las nuevas experiencias.

- La tasa de aprendizaje (α) controla qué tan rápido el agente actualiza sus valores Q ante nueva información; un valor elevado puede acelerar la convergencia inicial pero generar inestabilidad, mientras que un valor bajo asegura una convergencia más suave y robusta.
- El parámetro épsilon (ϵ) determina el grado de exploración, permitiendo que el agente descubra nuevas rutas hacia el pasajero, mientras que el decaimiento (decay) es una estrategia crucial para reducir gradualmente esta exploración y permitir que el agente explore su conocimiento experto conforme avanza el entrenamiento.
- El factor de descuento (γ) dicta la importancia de las recompensas futuras frente a las inmediatas; un γ cercano a 1.0 es vital para que el taxista priorice completar la entrega final (recompensa de +20) a pesar de los costos negativos acumulados en cada paso del trayecto.

```
[14]: # @title Aprendizaje on-policy  
agent_S = SARSAAgent(env, epsilon=0.01, alpha=0.1, discount_factor=1.0, ▮  
    ↪decay=False)  
agent_S.train(num_episodes=100000)  
list_stats_S_epsilon, list_lengths_S_epsilon = agent_S.stats()  
Q_S = agent_S.q_values
```

```
10%|          | 10355/100000 [00:04<00:25, 3466.66it/s]  
success: -4.3931, epsilon: 0.0100  
21%|          | 20635/100000 [00:07<00:22, 3536.94it/s]  
success: 1.5248, epsilon: 0.0100  
31%|          | 30514/100000 [00:10<00:19, 3531.85it/s]  
success: 3.4939, epsilon: 0.0100  
41%|          | 40559/100000 [00:13<00:16, 3601.37it/s]  
success: 4.4671, epsilon: 0.0100  
50%|          | 50483/100000 [00:16<00:14, 3382.00it/s]
```



```

success: 5.0608, epsilon: 0.0100
61%|      | 60523/100000 [00:19<00:11, 3539.02it/s]
success: 5.4646, epsilon: 0.0100
71%|      | 70543/100000 [00:21<00:08, 3561.54it/s]
success: 5.7529, epsilon: 0.0100
81%|      | 80598/100000 [00:24<00:05, 3448.50it/s]
success: 5.9627, epsilon: 0.0100
91%|      | 90624/100000 [00:27<00:02, 3599.24it/s]
success: 6.1260, epsilon: 0.0100
100%|     | 100000/100000 [00:30<00:00, 3287.43it/s]
success: 6.2553, epsilon: 0.0100

```

```

[15]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=0.2, alpha=0.5, discount_factor=1.0,
    ↪decay=True)
agent_S.train(num_episodes=100000)
list_stats_S_alpha, list_lengths_S_alpha = agent_S.stats()
Q_S = agent_S.q_values

```

```

10%|      | 10248/100000 [00:09<00:36, 2436.50it/s]
success: -113.8980, epsilon: 0.1000
20%|      | 20419/100000 [00:13<00:28, 2831.35it/s]
success: -56.7255, epsilon: 0.0500
30%|      | 30423/100000 [00:16<00:22, 3074.03it/s]
success: -36.6071, epsilon: 0.0333
40%|      | 40393/100000 [00:19<00:19, 3003.46it/s]
success: -26.2814, epsilon: 0.0250
51%|      | 50545/100000 [00:22<00:14, 3369.32it/s]
success: -19.9377, epsilon: 0.0200
61%|      | 60512/100000 [00:25<00:12, 3153.95it/s]
success: -15.6118, epsilon: 0.0167
71%|      | 70581/100000 [00:28<00:08, 3335.88it/s]
success: -12.4836, epsilon: 0.0143
80%|      | 80494/100000 [00:31<00:05, 3447.88it/s]

```

```

success: -10.1287, epsilon: 0.0125
90%|      | 90456/100000 [00:34<00:02, 3459.51it/s]
success: -8.2759, epsilon: 0.0111
100%|     | 100000/100000 [00:37<00:00, 2646.84it/s]
success: -6.8027, epsilon: 0.0100

```

```

[16]: # @title Aprendizaje on-policy
agent_S = SARSAAgent(env, epsilon=0.2, alpha=0.1, discount_factor=0.25,
    ↪decay=True)
agent_S.train(num_episodes=100000)
list_stats_S_discount, list_lengths_S_discount = agent_S.stats()
Q_S = agent_S.q_values

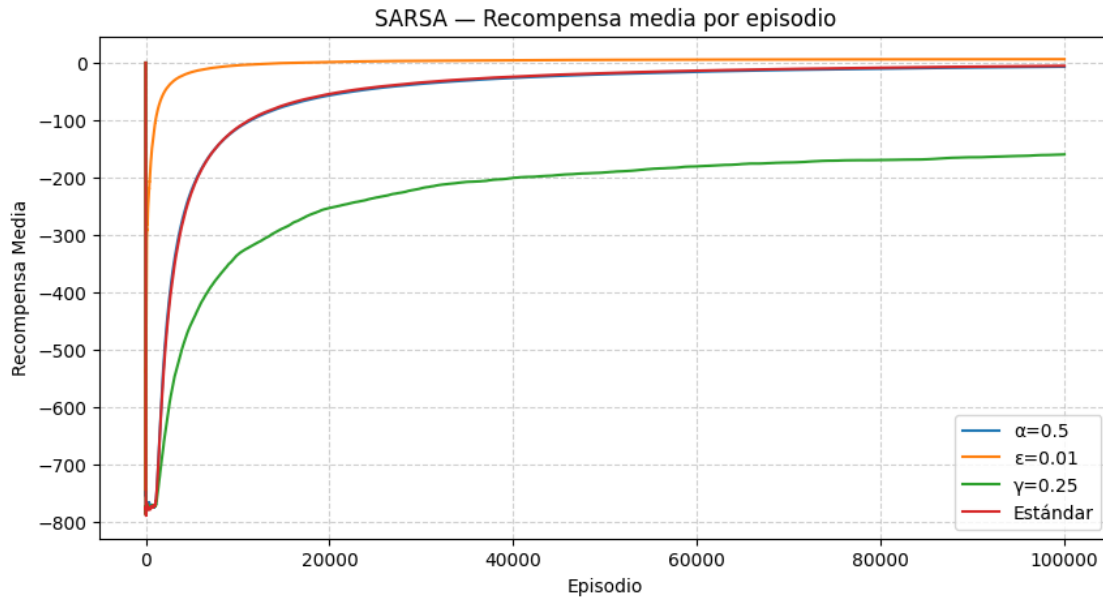
```

```

10%|      | 10042/100000 [00:33<04:31, 331.82it/s]
success: -334.5742, epsilon: 0.1000
20%|      | 20054/100000 [01:03<04:20, 306.35it/s]
success: -252.8930, epsilon: 0.0500
30%|      | 30067/100000 [01:33<02:12, 527.55it/s]
success: -218.5159, epsilon: 0.0333
40%|      | 40060/100000 [02:01<02:29, 399.98it/s]
success: -200.5392, epsilon: 0.0250
50%|      | 50056/100000 [02:30<02:04, 400.89it/s]
success: -190.9456, epsilon: 0.0200
60%|      | 60075/100000 [02:55<01:44, 383.08it/s]
success: -180.4032, epsilon: 0.0167
70%|      | 70056/100000 [03:22<01:29, 334.35it/s]
success: -173.6606, epsilon: 0.0143
80%|      | 80039/100000 [03:49<00:55, 361.61it/s]
success: -169.3313, epsilon: 0.0125
90%|      | 90049/100000 [04:14<00:29, 339.76it/s]
success: -164.5310, epsilon: 0.0111
100%|     | 100000/100000 [04:37<00:00, 359.81it/s]
success: -159.6134, epsilon: 0.0100

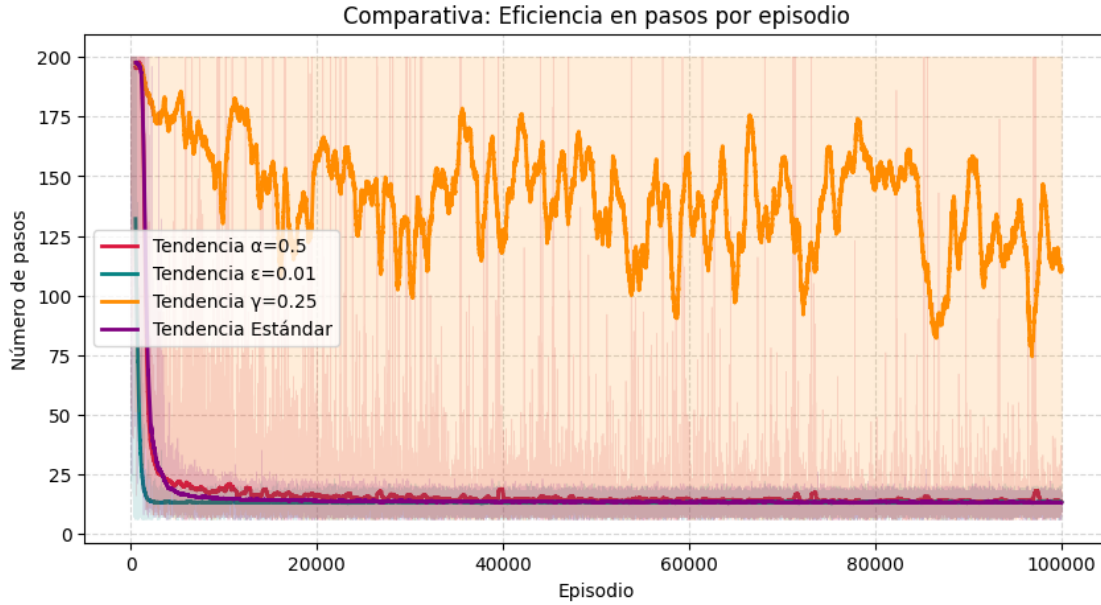
```

```
[17]: #@title Recompensa media por episodio (on-policy)
lista_plots_recompensa = [list_stats_S_alpha, list_stats_S_epsilon,
    ↳ list_stats_S_discount, list_stats_S]
labels = ['=0.5', '=0.01', '=0.25', 'Estándar']
plot_comparison(lista_plots_recompensa, title='SARSA - Recompensa media por
    ↳ episodio', labels=labels)
for lista, label in zip(lista_plots_recompensa, labels):
    print(f'Recompensa media final de {label}: {lista[-1]:.2f}')
```



Recompensa media final de =0.5: -6.80
 Recompensa media final de =0.01: 6.26
 Recompensa media final de =0.25: -159.61
 Recompensa media final de Estándar: -5.34

```
[18]: #@title Longitud de episodios (on-policy)
lista_plots_longitud = [list_lengths_S_alpha, list_lengths_S_epsilon,
    ↳ list_lengths_S_discount, list_lengths_S]
plot_lengths_comparison(lista_plots_longitud, labels=labels)
for lista, label in zip(lista_plots_longitud, labels):
    print(f'Longitud media final (últimos 1000 episodios) de {label}: {np.
    ↳ mean(lista[-1000:]):.2f} pasos')
```



Longitud media final (últimos 1000 episodios) de $\alpha=0.5$: 13.80 pasos
Longitud media final (últimos 1000 episodios) de $\epsilon=0.01$: 13.25 pasos
Longitud media final (últimos 1000 episodios) de $\gamma=0.25$: 112.51 pasos
Longitud media final (últimos 1000 episodios) de Estándar: 13.04 pasos

1.4.4 3.3.1 Análisis de los hiperparámetros

1.4.5 3.3.1 Análisis de Resultados y Comparación de Configuraciones

En esta sección se evalúa el impacto de los hiperparámetros en el aprendizaje de SARSA. Se han definido tres configuraciones específicas (más la original) para observar los distintos hiperparámetros afectan en el desarrollo del entrenamiento en el entorno Taxi-v3:

1. **Configuración 1** ($\epsilon = 0.01$) sin decaimiento: Minimiza la exploración desde el inicio, lo que puede llevar a una convergencia más rápida pero también a un riesgo de quedar atrapado en políticas subóptimas debido a la falta de descubrimiento de rutas alternativas.
2. **Configuración 2** ($\alpha = 0.5$): Aumenta la tasa de aprendizaje, lo que puede acelerar la convergencia inicial pero también introduce más ruido en las actualizaciones, potencialmente causando oscilaciones en el rendimiento antes de estabilizarse.
3. **Configuración 3** ($\gamma = 0.25$): Reduce la importancia de las recompensas futuras, lo que puede hacer que el agente se enfoque más en obtener recompensas inmediatas, pero también puede dificultar la convergencia a la política óptima que requiere una planificación a largo plazo.

Interpretación de las Gráficas A partir de las dos gráficas de comparación generadas (plot_comparison y plot_lengths_comparison), se extraen las siguientes conclusiones:

Recompensa Media Acumulada: - La peor configuración es la que implica reducir la tasa de descuento. Esto implica que la recompensa a largo plazo se difumina y se opte por estrategias de

recompensas a corto plazo. Con esta tasa el algoritmo no converge (recompensa media final de -159.61).

- El otorgar mayor peso a los nuevos datos (mayor α) hace que el algoritmo funcione algo peor respecto al estándar. Esto implica un valor de 0.2 es adecuado y que se prefiere confiar más en los datos que ya se han obtenido.
- La recompensa media final de la configuración con un epsilon fijo a 0.01 es de 6.26, lo que supera a los valores originales. Por otro lado, el algoritmo converge antes según se ve en la gráfica. Todo esto se debe a que con la configuración inicial, epsilon empieza siendo 1 durante los primeros 1000 episodios, por lo que las acciones son completamente aleatorias. En cambio, con un epsilon fijo a 0.01, el agente explora un poco pero principalmente explota desde el inicio, lo que le permite aprender rápidamente una política decente. Para el entorno de Taxi-v3, se valora más la explotación temprana que la exploración extensa, ya que el espacio de estados es relativamente pequeño y solo hay una recompensa positiva significativa al completar la tarea. Dicho de otra manera, al no existir ‘recompensas secundarias’ que actúen como máximos locales, el agente no se ve distraído.

Longitud de los Episodios: - La menor longitud de pasos en la configuración Estándar (aunque mínima) demuestra que una mayor tasa de exploración inicial permite al algoritmo descubrir rutas más eficientes y cercanas al óptimo global. Sin embargo, su recompensa media es inferior debido a que mantiene un factor de exploración (ϵ) más alto durante el entrenamiento. En SARSA, esto se traduce en penalizaciones frecuentes por acciones aleatorias (como intentos de descarga ilegales), las cuales hunden el promedio de recompensa a pesar de que la política de navegación aprendida es superior en distancia a la del agente de $\epsilon = 0.01$.

1.5 4. Análisis y Conclusiones

1.5.1 4.1 Comparativa de Desempeño frente a Monte Carlo

El algoritmo de SARSA es capaz de converger a una política óptima en Taxi-v3 mucho más rápido que el método de Monte Carlo Off-Policy. Mientras que MC requiere episodios completos para actualizar la tabla Q , SARSA actualiza en cada paso, lo que permite aprender a evitar penalizaciones severas (-10 por acciones ilegales) desde los primeros episodios. La longitud media final de pasos es de 13, lo que indica que el agente ha aprendido a realizar la secuencia correcta de acciones para completar la tarea eficientemente, ya que estamos en un mundo con 25 casillas (grid de 5x5).

Por otro lado, la recompensa media final es de 5.34, reflejando la eficiencia del agente para completar entregas exitosas pero siendo lastrado por su comportamiento epsilon-greedy que le fuerza a tomar decisiones aleatorias (como un dropout ilegal), ya que cada paso es una recompensa de -1 pero entregar al pasajero correctamente otorga +20.

La tabla de los valores esperados de cada estado-acción (Q) muestra que se han visitado todos las posibilidades (2400). En la documentación del escenario (https://gymnasium.farama.org/environments/toy_text/taxi/) se indica que el número de estados alcanzables en un episodio es de 400 sobre el total de 500. Esto ocurre porque existen escenarios donde el pasajero tiene como lugar de origen el de destino, lo cual indica la finalización del episodio. De esta manera, 400 estados por 6 acciones por estado equivalen a los 2400 estados-acción visitados.

Los valores mínimo, máximo y medio para la tabla Q refuerzan la hipótesis de que SARSA es

un algoritmo conservador. Al ser un algoritmo on-policy, existe la posibilidad de que al explorar distintas acciones, cometa acciones ilegales, penalizadas con una recompensa de -10 y por eso el valor medio de Q no se acerca más a 0. El valor mínimo de Q refleja un estado donde el agente ha sufrido múltiples penalizaciones.

Por último, al observar un episodio greedy, se confirma que el agente ha aprendido la secuencia lógica de acciones: primero se dirige al pasajero, luego realiza el pickup, después se dirige al destino y finalmente hace el dropoff. Esto lo hace sin cometer errores (deambular sin rumbo o chocar con muros), lo que demuestra la efectividad del aprendizaje de SARSA en este entorno. Sumado a esto, si vemos la visualización de que acción es la preferida en cada estado, se confirma que el agente ha aprendido a evitar muros y a dirigirse correctamente hacia el pasajero y el destino.

1.5.2 4.2 ¿Por qué SARSA funciona mejor que Monte Carlo?

Afirmamos que SARSA funciona mejor que Monte Carlo debido a su velocidad de convergencia hacia la política óptima (longitud media de pasos por episodio) y su recompensa media final más alta.

Una de las razones por la que SARSA funciona mejor es que al actualizar la tabla Q en cada paso, SARSA no necesita terminar el episodio para aprender. En los primeros episodios (donde el taxi vaga sin rumbo), esto permite aprender rápidamente que chocar con muros o realizar “pickups” ilegales es malo.

Por otro lado, Monte Carlo solo actualiza al final del episodio, por lo que en episodios largos, que se dan al principio del aprendizaje, el agente no recibe retroalimentación hasta que se completa la tarea o se alcanza el límite de pasos. Esto hace que el aprendizaje sea mucho más lento, especialmente en un entorno con penalizaciones severas como Taxi-v3. Además, al usar MC Off-policy, si el agente ejecuta una acción que no es la óptima, se desecha el episodio completo, lo que retrasa aún más el aprendizaje.

1.5.3 4.3 Propuestas Futuras

- Implementar **Q-Learning** para comparar si una política puramente Off-Policy logra una convergencia aún más rápida al ignorar el coste de la exploración.
- Evaluar el impacto de diferentes tasas de decaimiento de ϵ en la estabilidad de la tabla Q .